

Taller en parejas: Muro de ideas

Guía de la pareja — versión Django · Un mini proyecto real del curso de Desarrollo de Aplicaciones Web, UPB

¿Qué van a construir?

Una **aplicación web completa** de dos capas, igual que las del curso:

Capa	Qué es	Quién la conduce
Backend (Python + Django)	Un servidor con una API REST que guarda y entrega las ideas en formato JSON	Estudiante universitario
Frontend (HTML + CSS + JavaScript)	La página que ve el usuario, que le pide los datos al servidor con <code>fetch</code>	Estudiante de colegio

El resultado es un **muro de ideas**: se publica una idea, se le da «me gusta» y se puede borrar. Esas cuatro acciones son un **CRUD** (Crear, Leer, Actualizar, Borrar), la base de casi cualquier aplicación.

```
Navegador (frontend)  --- fetch ---> Servidor Django (backend)
HTML + CSS + JS       <--- JSON  <--- API REST en /api/ideas
```

Cómo trabajan en pareja

- **Piloto**: quien teclea. **Copiloto**: quien lee la guía y revisa.
- **Cambien de rol en cada parte**. Quien conduce cada parte está indicado abajo.
- El universitario **explica**, no teclea por el compañero. El de colegio **pregunta** todo lo que no entienda.
- Usen un **apodo** y no escriban datos privados en el muro.

Conceptos clave (léanlos juntos, 5 min)

API: una forma en que dos programas se hablan. Aquí, la página (frontend) le *pide cosas* al servidor (backend) por direcciones como `/api/ideas`.

JSON: el formato en que viajan los datos. Una idea se ve así:

```
{ "id": 1, "autor": "Ana", "texto": "Un club de robótica", "likes": 0 }
```

Métodos HTTP: el «verbo» de cada petición.

Acción	Método	Dirección	Respuesta correcta
Leer todas	GET	<code>/api/ideas</code>	200 OK
Crear una	POST	<code>/api/ideas</code>	201 Created
Dar like	PUT	<code>/api/ideas/2/like</code>	200 OK
Borrar	DELETE	<code>/api/ideas/2</code>	204 No Content

Códigos de estado: `400` = la petición venía mal (faltan datos). `404` = no existe lo que se pidió.

fetch: la función de JavaScript con la que la página hace peticiones al servidor.

Parte 0 — Preparación (10 min)

Si su facilitador ya les entregó el proyecto base `muro-django`, ábralo en VS Code o PyCharm, activen el entorno virtual y salten al paso 0.6.

1. Creen una carpeta llamada `muro-django` y ábranla en **VS Code** (File → Open Folder). Abran la terminal (**Terminal** → **New Terminal**).
2. Creen y activen un entorno virtual:

```
python -m venv venv
venv\Scripts\activate
```

(en Mac/Linux: `source venv/bin/activate`; si `python` no se reconoce, prueben `python3`)

3. Instalen Django y creen el proyecto y la aplicación (**no olviden el punto** al final del primer comando):

```
pip install django
django-admin startproject muro .
python manage.py startapp ideas
```

4. En `muro/settings.py`, agreguen `"ideas"` a la lista `INSTALLED_APPS`:

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "ideas",
]
```

5. Creen una carpeta llamada `public` al lado de `manage.py` (aquí vivirá el frontend).
6. Verifiquen que todo arranca:

```
python manage.py runserver
```

Debes ver: en la consola `Starting development server at http://127.0.0.1:8000/`. Al abrir `http://localhost:8000` aparece la página de bienvenida de Django con el cohete. Un aviso sobre «migraciones sin aplicar» es normal: hoy no usamos base de datos. Detengan el servidor con **Ctrl + C**.

Parte 1 — El servidor: primera versión (20 min)

Conduce: universitario · Copiloto: colegial

Hay que tocar tres archivos.

1. `ideas/views.py` (reemplacen todo el contenido): las vistas que atienden las peticiones.

```

import json

from django.http import HttpResponse, JsonResponse
from django.views.decorators.csrf import csrf_exempt
from django.views.decorators.http import require_http_methods

# "Base de datos" en memoria: se reinicia al apagar el servidor
ideas_lista = [
    {'id': 1, 'autor': 'Profe', 'texto': '¡Bienvenidos al muro de ideas!', 'likes': 0}
]
siguiente_id = 2

def buscar(id):
    for idea in ideas_lista:
        if idea['id'] == id:
            return idea
    return None

# Listar todas las ideas
@csrf_exempt
@require_http_methods(['GET'])
def ideas(request):
    return JsonResponse(ideas_lista, safe=False)

```

`@csrf_exempt` desactiva la protección CSRF para que el `fetch` del frontend funcione sin token. En el curso verán cómo protegerse del CSRF más adelante.

2. `ideas/urls.py` (archivo **nuevo**, créenlo dentro de la carpeta `ideas`): las rutas de la API.

```

from django.urls import path
from . import views

urlpatterns = [
    path('ideas', views.ideas),
]

```

3. `muro/urls.py` (reemplacen todo el contenido): conecta la API y sirve el frontend.

```

from django.conf import settings
from django.urls import include, path, re_path
from django.views.static import serve

# Carpeta donde vive el frontend (index.html, style.css, app.js)
PUBLIC = settings.BASE_DIR / 'public'

urlpatterns = [
    path('api/', include('ideas.urls')),
    path('', serve, {'path': 'index.html', 'document_root': PUBLIC}),
    re_path(r'^(?P<path>(style\.css|app\.js))$', serve, {'document_root': PUBLIC}),
]

```

Enciendan el servidor:

```
python manage.py runserver
```

Debes ver: Starting development server at `http://127.0.0.1:8000/`. Abran en el navegador

`http://localhost:8000/api/ideas`: aparece el JSON de la idea del Profe (las tildes se ven como `\u00a1`, es normal en JSON).
¡Ya tienen una API funcionando!

Preguntas para el universitario: ¿qué relación hay entre `urls.py` y `views.py`? ¿Qué hace `include`? ¿Por qué se llama «en memoria» esta base de datos?

Regla de oro: Django **se reinicia solo** cada vez que guardan un archivo `.py`. Si algo falla, miren primero la terminal. Los cambios del frontend solo necesitan recargar la página (F5).

Parte 2 — La página: HTML y CSS (25 min)

Conduce: colegial · Copiloto: universitario

Aquí brilla lo que ya saben de HTML. Creen `public/index.html`:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Muro de ideas</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <h1>Muro de ideas</h1>
    <p>Comparte una idea corta con el curso</p>
  </header>

  <main>
    <section id="formulario">
      <h2>Nueva idea</h2>
      <form id="form-idea">
        <input type="text" id="autor" placeholder="Tu apodo" maxlength="20" required>
        <textarea id="texto" placeholder="Escribe tu idea (máx. 140 caracteres)" maxlength="140" required>
      </textarea>
      <button type="submit">Publicar</button>
    </form>
    <p id="mensaje"></p>
  </section>

  <section id="muro">
    <h2>Ideas publicadas</h2>
    <div id="lista"></div>
  </section>
</main>

<script src="app.js"></script>
</body>
</html>
```

Y `public/style.css`. Pueden cambiar los colores y el estilo como quieran; es su diseño:

```
* { box-sizing: border-box; }

body {
  font-family: Arial, sans-serif;
```

```

    margin: 0;
    background: #f4f4f9;
    color: #222;
}

header {
    background: #4f46e5;
    color: white;
    text-align: center;
    padding: 1.5rem 1rem;
}

header h1 { margin: 0 0 0.3rem; }
header p { margin: 0; }

main {
    display: flex;
    flex-wrap: wrap;
    gap: 1.5rem;
    justify-content: center;
    align-items: flex-start;
    padding: 2rem 1rem;
}

#formulario, #muro {
    background: white;
    border-radius: 12px;
    padding: 1.5rem;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.15);
}

#formulario { width: 320px; }
#muro { width: 460px; max-width: 100%; }

form { display: flex; flex-direction: column; gap: 0.7rem; }

input, textarea {
    font-family: inherit;
    font-size: 1rem;
    padding: 0.6rem;
    border: 1px solid #ccc;
    border-radius: 8px;
}

textarea { min-height: 90px; resize: vertical; }

button {
    background: #4f46e5;
    color: white;
    border: none;
    padding: 0.6rem 1rem;
    border-radius: 8px;
    font-size: 1rem;
    cursor: pointer;
}

button:hover { background: #3730a3; }

#mensaje { color: #b91c1c; min-height: 1.2rem; }

.idea {
    border-left: 5px solid #4f46e5;
    background: #f9fafb;

```

```
padding: 0.8rem 1rem;
border-radius: 8px;
margin-bottom: 0.8rem;
}

.idea .autor { font-weight: bold; margin: 0 0 0.3rem; }
.idea .texto { margin: 0 0 0.6rem; overflow-wrap: anywhere; }

.acciones { display: flex; gap: 0.5rem; }
.acciones button { padding: 0.35rem 0.8rem; font-size: 0.9rem; }
.acciones .borrar { background: #dc2626; }
.acciones .borrar:hover { background: #991b1b; }
```

Con el servidor encendido abran **http://localhost:8000** (¡ya no el archivo directamente!).

Debes ver: el encabezado morado, el formulario a la izquierda y el cuadro «Ideas publicadas» vacío a la derecha.

¿Por qué se abre desde localhost:8000? Porque en `muro/urls.py` le dijimos a Django que sirva los archivos de la carpeta `public` con `django.views.static.serve`. Así, la página y la API viven en la misma dirección (`localhost:8000`) y todo se conecta sin problemas. *(En un proyecto real, los archivos estáticos se manejan con `STATICFILES_DIRS`, como vieron en clase; aquí usamos un atajo para que el mismo frontend funcione en todos los stacks.)*

Parte 3 — Conectar frontend y backend (25 min)

3.1 Leer las ideas: `fetch` con GET

Conduce: **colegial**

Creen `public/app.js`:

```

const lista = document.getElementById('lista');
const mensaje = document.getElementById('mensaje');
const form = document.getElementById('form-idea');

// Dibuja una idea en pantalla.
// Usamos textContent (y no innerHTML) para que nadie pueda inyectar código en el muro.
function crearTarjeta(idea) {
  const tarjeta = document.createElement('div');
  tarjeta.className = 'idea';

  const autor = document.createElement('p');
  autor.className = 'autor';
  autor.textContent = idea.autor;

  const texto = document.createElement('p');
  texto.className = 'texto';
  texto.textContent = idea.texto;

  tarjeta.append(autor, texto);
  return tarjeta;
}

// READ: pedir todas las ideas al backend
async function cargarIdeas() {
  const respuesta = await fetch('/api/ideas');
  const ideas = await respuesta.json();
  lista.innerHTML = '';
  ideas.forEach(idea => lista.appendChild(crearTarjeta(idea)));
}

cargarIdeas();

```

Recarguen la página (F5).

Debes ver: la tarjeta de «Profe» con «¡Bienvenidos al muro de ideas!». **Esa información viene del servidor, no del HTML.**

3.2 Crear ideas: POST en el backend

Conduce: universitario

En `ideas/views.py`, reemplacen la función `ideas` completa por esta versión, que atiende GET y POST:

```

# Listar y crear ideas
@csrf_exempt
@require_http_methods(['GET', 'POST'])
def ideas(request):
    global siguiente_id

    # Listar todas las ideas
    if request.method == 'GET':
        return JsonResponse(ideas_lista, safe=False)

    # Crear una idea
    try:
        datos = json.loads(request.body)
    except json.JSONDecodeError:
        return JsonResponse({'error': 'JSON inválido'}, status=400)

    autor = datos.get('autor')
    texto = datos.get('texto')
    if not autor or not texto:
        return JsonResponse({'error': 'autor y texto son obligatorios'}, status=400)
    if len(texto) > 140:
        return JsonResponse({'error': 'el texto no puede pasar de 140 caracteres'}, status=400)

    nueva = {'id': siguiente_id, 'autor': autor, 'texto': texto, 'likes': 0}
    siguiente_id += 1
    ideas_lista.append(nueva)
    return JsonResponse(nueva, status=201)

```

Guarden el archivo. Django se reinicia solo.

`json.loads(request.body)` convierte el JSON que llega en un diccionario de Python.

3.3 Crear ideas: el formulario envía con fetch

Conduce: colegial

En `app.js`, antes de la última línea `cargarIdeas();`, agreguen:

```
// CREATE: enviar una idea nueva
form.addEventListener('submit', async (evento) => {
  evento.preventDefault();
  mensaje.textContent = '';

  const respuesta = await fetch('/api/ideas', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      autor: document.getElementById('autor').value,
      texto: document.getElementById('texto').value
    })
  });

  if (!respuesta.ok) {
    const error = await respuesta.json();
    mensaje.textContent = error.error;
    return;
  }

  document.getElementById('texto').value = '';
  cargarIdeas();
});
```

Recarguen (F5), escriban un apodo y una idea y pulsen **Publicar**.

Debes ver: la idea aparece en el muro. Recarguen la página: **sigue ahí** porque quedó guardada en el servidor. Si apagan y encienden el servidor, vuelve a quedar solo la del Profe.

Para pensar: `preventDefault()` evita que el formulario recargue la página. ¿Qué pasaría si lo quitan? Pruébenlo.

Parte 4 — Me gusta y borrar: el CRUD completo (25 min)

4.1 Backend: PUT y DELETE

Conduce: universitario

1. Al final de `ideas/views.py` agreguen las dos vistas nuevas:

```

# Dar like a una idea
@csrf_exempt
@require_http_methods(['PUT'])
def like(request, id):
    idea = buscar(id)
    if idea is None:
        return JsonResponse({'error': 'idea no encontrada'}, status=404)
    idea['likes'] += 1
    return JsonResponse(idea)

# Eliminar una idea
@csrf_exempt
@require_http_methods(['DELETE'])
def detalle(request, id):
    idea = buscar(id)
    if idea is None:
        return JsonResponse({'error': 'idea no encontrada'}, status=404)
    ideas_lista.remove(idea)
    return HttpResponse(status=204)

```

2. En `ideas/urls.py` agreguen las dos rutas nuevas dentro de la lista:

```

urlpatterns = [
    path('ideas', views.ideas),
    path('ideas/<int:id>/like', views.like),
    path('ideas/<int:id>', views.detalle),
]

```

Guarden ambos archivos.

4.2 Frontend: botones

Conduce: colegial

En `app.js`, **reemplacen** la función `crearTarjeta` completa por esta versión con botones:

```
function crearTarjeta(idea) {
  const tarjeta = document.createElement('div');
  tarjeta.className = 'idea';

  const autor = document.createElement('p');
  autor.className = 'autor';
  autor.textContent = idea.autor;

  const texto = document.createElement('p');
  texto.className = 'texto';
  texto.textContent = idea.texto;

  const acciones = document.createElement('div');
  acciones.className = 'acciones';

  const btnLike = document.createElement('button');
  btnLike.textContent = '👍 ' + idea.likes;
  btnLike.addEventListener('click', () => darLike(idea.id));

  const btnBorrar = document.createElement('button');
  btnBorrar.className = 'borrar';
  btnBorrar.textContent = 'Borrar';
  btnBorrar.addEventListener('click', () => borrar(idea.id));

  acciones.append(btnLike, btnBorrar);
  tarjeta.append(autor, texto, acciones);
  return tarjeta;
}
```

Y antes de la última línea `cargarIdeas()`; agreguen las dos funciones nuevas:

```
// UPDATE: dar like
async function darLike(id) {
  await fetch('/api/ideas/' + id + '/like', { method: 'PUT' });
  cargarIdeas();
}

// DELETE: borrar una idea
async function borrar(id) {
  await fetch('/api/ideas/' + id, { method: 'DELETE' });
  cargarIdeas();
}
```

Recarguen (F5).

Debes ver: cada idea con un botón «👍 0» y uno rojo «Borrar». El like sube el contador y el botón borrar elimina la tarjeta. ¡Su CRUD está completo!

Parte 5 — Prueben, muestren y retos

Prueba de seguridad: escriban `<b>hola</b>` como idea. ¿Se muestra el texto tal cual o se pone en negrita? Pregúntenle al universitario por qué se usa `textContent` en lugar de `innerHTML` (ataque XSS, tema del curso).

Prueba de errores: abran `http://localhost:8000/api/ideas/99` en el navegador. ¿Qué pasa? Piensen qué código de estado debería responder la API.

Retos si les sobra tiempo

1. **Personalicen** el diseño: colores, tipografía, un título propio para el muro.
2. **Ordenar:** muestren primero la idea con más likes (pista: `ideas.sort(...)` en `cargarIdeas`).

3. **Contador de caracteres** que baje mientras escriben (evento `input` en el textarea).
4. **Campo nuevo:** una categoría (tecnología, deporte, arte...) en el backend y en el formulario.
5. **Nivel curso:** Reemplazar la lista `ideas_lista` por una base de datos **MongoDB Atlas** con MongoEngine, como hacen en el curso.

Si algo no funciona

Síntoma	Causa probable	Solución
<code>ModuleNotFoundError: No module named 'django'</code>	El entorno virtual no está activado	Activen <code>venv</code> (verán <code>(venv)</code> al inicio de la terminal)
En PowerShell no deja activar <code>venv</code>	La política de scripts lo bloquea	Usen la terminal «Command Prompt» o pidan ayuda al facilitador
<code>Page not found (404)</code> en <code>/</code>	<code>public</code> mal ubicada o <code>muro/urls.py</code> incompleto	<code>public</code> debe estar al lado de <code>manage.py</code> ; revisen <code>muro/urls.py</code>
<code>Page not found</code> en <code>/api/ideas/</code>	Escribieron la barra final	La ruta es <code>/api/ideas</code> (sin barra al final)
<code>403 Forbidden (CSRF)</code> al publicar	Falta <code>@csrf_exempt</code> en la vista	Revisen los decoradores sobre la función
<code>405 Method Not Allowed</code>	La vista no incluye ese método	Revisen la lista de <code>@require_http_methods</code>
<code>NameError</code> o <code>AttributeError</code>	Código pegado fuera de lugar o sin guardar	Revisen la terminal: Django indica el archivo y la línea
<code>Error: That port is already in use</code>	Otro servidor encendido en el puerto 8000	Cierren la otra terminal o usen <code>runserver 8001</code>
Al publicar sale <code>400</code>	Llegan vacíos <code>autor</code> o <code>texto</code>	Revisen que los <code>id</code> del HTML coincidan con los del JS
Error al abrir <code>index.html</code> con doble clic	Se abrió el archivo, no el servidor	Abran siempre <code>http://localhost:8000</code>

Herramienta clave: **F12** en el navegador. En **Console** ven los errores de JavaScript y en **Network** ven cada petición con su método y su código de estado.

¿Qué sigue?

Lo que hicieron hoy es lo que se hace en Desarrollo de Aplicaciones Web: un backend con **API REST**, un frontend que lo consume y las operaciones **CRUD**. En el curso se suma:

- **Bases de datos reales** (MongoDB Atlas) para que los datos no se pierdan.
- **Documentación de la API** con Swagger.
- **Frameworks modernos** de frontend: React, Vue y Angular.
- Otros backends: Express, Spring Boot y .NET.

Si les gustó, **sigan practicando:** todo programador empezó con un proyecto pequeño como este.